

```

from kivy.uix.videoPlayer import VideoPlayer
from kivy.app import App

class HelloApp(App):
    def build(self):
        player=VideoPlayer(source="F:\Movie\j1.mp4",state="play",options={'eos':'loop'})
        return player
if __name__ == '__main__':
    HelloApp().run()
    
```

Figure 5: *main.py* file for video player

make. These sub-classes can be of the following types:

Layout	Description
GridLayout	Widgets are arranged in a grid. It has two properties, 'cols' and 'rows' and at least one should be specified. Its base class is <i>kivy.uix.gridlayout.GridLayout</i> class.
FloatLayout	Widgets are organised with arbitrary locations and sizes, either absolute or relative to window sizes. Its base class is <i>kivy.uix.floatlayout.FloatLayout</i> class.
RelativeLayout	It behaves similar to FloatLayout, but its position is relative to the Layout size and not to the window size. Its base class is <i>kivy.uix.relativelayout.RelativeLayout</i> .
AnchorLayout	It allows the positioning of children relative to borders of the layout. Its base class is <i>kivy.uix.anchorlayout.AnchorLayout</i> .
BoxLayout	It arranges children in horizontal or vertical boxes, i.e., in the form of one row or one column. Its base class is <i>kivy.uix.boxlayout.BoxLayout</i> .
StackLayout	It is similar to BoxLayout with a difference –its children can be positioned in multiple rows and columns when it runs out of space. Its base class is <i>kivy.uix.stacklayout.StackLayout</i> .



Figure 6: Video player playing video

The following Kivy Language code shows the use of layouts and properties:

```

BoxLayout:
    Button:
        text: 'Button'
        size_hint: 0.5, 0.5
        pos_hint: {"center_x": 0.5,"center_y":0.5}
    
```

A basic video player Kivy app

Let's use Kivy's video player widget to play video. We will make some minor changes to our 'hello world' example. Open *main.py* and add the following line at the top:

```
fromkivy.uix.videoPlayer import VideoPlayer
```

Now add the following Python function to the *main.py* file in place of the pass keyword:

```

def build(self):

    player=VideoPlayer(source="j1.mp4",state="play",options={'eos':'loop'})

    return player
    
```

The source should be given the path of the video you want to play. For more options, check out Kivy documentation.

Windows users can launch the application with the right-click method and Linux users can launch it from the command line. 

References

- [1] Kivy documentation, www.kivy.org/docs/
- [2] Kivy – Interactive Applications and Games in Python, Packt Publishing

By: Shubham Sharma

The author is an open source activist studying computer science engineering. He can be contacted at shubhamsharma@gmail.com / shubham@cyberzonec.in.